

PLANO DE TESTES — ESTEIRAS DE CI/CD (BACKEND, FRONTEND E MOBILE) 1

Identificação

- **Nome do plano:** Plano de Testes dos pipelines do GitHub Actions.
- **Sistema alvo:** Workflows do GitHub Actions do FaculRide (Backend, Frontend e Mobile).
- **Versão:** 1.0
- **Tipo de sistema:** CI/CD
- **Responsável pelos testes:** Equipe de Desenvolvimento.
- **Ambiente:** GitHub Actions e Ambientes de Homologação/Produção.

Objetivo

Este plano tem como objetivo definir a estratégia para validar os pipelines automatizados das três frentes do ecossistema FaculRide (Backend, Frontend e Mobile) antes das futuras implementações com o Azure.

A finalidade é verificar se a validação de mensagens de commit via commitlint, compilação (build), atualização do arquivo package.json, cálculo de versão semântica (Semver), criação de releases/tags, publicação de imagens Docker e envio de alertas por e-mail funcionam corretamente em todos os repositórios.

Escopo

Funcionalidades incluídas

Fazem parte do escopo desta etapa:

1. **Gatilhos por Branch:** Disparos automáticos em pushes nas branches develop e main em todas as frentes.
2. **Padronização de Commits:** Execução do commitlint (wagoid/commitlint-github-action) e plugins do Semantic Release.
3. **Versionamento Automático e Modificação de Arquivos:**
 - a. Cálculo de versão semântica e tags de preview (-preview.N) na branch develop.
 - b. Cálculo de incremento de produção (MAJOR, MINOR, PATCH) na branch main.
 - c. Edição dinâmica do package.json e auto-commit com a tag [skip ci] para evitar loops infinitos (Mobile).
4. **Containerização e Publicação (Docker Hub):**
 - a. **Backend:** Publicação no repositório do Docker Hub com as tags development-latest, development-vX.Y.Z, production-latest e production-vX.Y.Z.
 - b. **Frontend:** Publicação na conta goncahri/faculride-frontend com as tags development-latest, development-vX.Y.Z, production-latest e production-vX.Y.Z.
 - c. **Mobile:** Publicação no repositório de containers do Docker Hub com as tags development e production-vX.Y.Z.
5. **Gatilhos por Branch:** Disparos automáticos em pushes nas branches develop e main em todas as frentes.

Tratamento de Falhas: Envio de e-mails de notificação via dawidd6/action-send-mail quando o pipeline falhar em qualquer repositório.

Funcionalidades excluídas

Não fazem parte do escopo desta etapa:

- Regras de negócio, interfaces visuais ou testes funcionais internos das aplicações;

- Testes de infraestrutura ou falhas de conectividade diretas do GitHub ou Docker Hub.

Critérios de Entrada e Saída

Critérios de entrada

Os testes poderão ser iniciados quando:

- Os arquivos de workflow estiverem salvos e ativos nos três repositórios;
- A configuração das secrets estiver concluída nos repositórios.

Critérios de saída

Os testes poderão ser encerrados quando:

- Houver execução sem erros das tarefas planejadas em cada repositório;
- Ocorrer a criação bem-sucedida das releases e tags no GitHub;
- For verificada a atualização do package.json no histórico de commits (Mobile);
- As imagens do Backend, Frontend e Mobile forem publicadas no Docker Hub com as tags correspondentes;
- O disparo do e-mail de alerta for validado por meio de uma falha forçada.

Ambiente de Teste

Hardware

O ambiente utilizado será:

- Execuções virtuais hospedadas pelo GitHub;
- Acesso à internet.

Software

O ambiente poderá utilizar:

- Runtime necessário para execução do backend;
- Framework utilizado no frontend;
- Containers Docker.

Ferramentas

Podem ser utilizadas ferramentas como:

- **GitHub Actions:** ambiente de execução;
- **Docker Hub:** para as imagens do projeto;
- **SMTP Mailer:** para notificação de falha do workflow;
- **Git/GitHub:** controle de versão;
- **Ferramenta de teste de carga:** avaliação de desempenho.

Recursos

- **Analista de Testes / QA:** Elaborar e executar os casos de teste, validar artefatos gerados e produzir o relatório final.
- **Desenvolvedor:** Corrigir falhas nos scripts YAML dos workflows e apoiar a equipe durante a investigação de erros.
- **Analista de Requisitos / Líder:** Esclarecer requisitos da esteira de CI/CD e apoiar a análise dos resultados obtidos.

Cronograma

- **Fase 1:** Mapeamento de requisitos e cenários de execução das três aplicações;
- **Fase 2:** Execução dos testes de Integração Contínua em branch de testes (Backend, Frontend e Mobile);
- **Fase 3:** Execução dos testes de Entrega nas branches main e develop (Semver, Tags e Docker Hub);
- **Fase 4:** Simulação de quebra de pipeline para validação do job `notify_failure()`;

- **Fase 5:** Análise de logs, consolidação de métricas e elaboração do Relatório Final.

Riscos

- **Loop infinito de Commits no Mobile:** O envio automático de alterações no package.json pode disparar o pipeline novamente.
 - *Mitigação:* Garantir a presença da tag `[skip ci]` na mensagem do commit gerado pelo bot do GitHub.
- **Falha no cálculo do Semver (Mobile e Backend):** Erros de sintaxe ao obter tags anteriores.
 - *Mitigação:* Uso do flag `--prune` no `git fetch` e fallback de versão inicial `v0.0.0`.
- **Divergência de versões do Node.js:** Problemas de compatibilidade em dependências.
 - *Mitigação:* Fixação das versões do Node.js nos workflows.

Métricas

- **Taxa de Sucesso da Esteira:** $(\text{Número de pipelines concluídos com êxito} / \text{Total de execuções}) \times 100$.
- **Taxa de Rejeição de Commits:** Percentual de commits bloqueados pela ação do commitlint por descumprirem o padrão.
- **Tempo Médio de Build e Push:** Tempo médio gasto em cada ecossistema (Backend vs Frontend vs Mobile) para construir a imagem Docker e enviá-la ao Docker Hub.

Resultado Esperado

Comprovar que os pipelines de CI/CD do Backend, Frontend e Mobile funcionam de forma consistente, garantindo o versionamento correto e a disponibilização dos artefatos no Docker Hub. O relatório final apresentará:

- Confirmação da criação de versões de testes (preview) e releases oficiais;

- Evidências da atualização do arquivo package.json pelo bot e validação do bloqueio de loop via [skip ci] no aplicativo Mobile;
- Confirmação da publicação das imagens com suas respectivas tags na plataforma Docker Hub;
- Comprovação do recebimento de e-mails de notificação sempre que ocorrer uma falha durante as execuções.